# Introduction to Computer Modeling

*Lecture Notes*

*Ivane Murusidze*

## 1. Introduction

*Computer modeling* of a system, process or phenomenon, involves the creation of a version of the *mathematical model* of the system (process, phenomenon) in the form of a *computer program*. *Computer simulations* that are run according to such programs produce knowledge about the system and its behavior that is usually out of reach of mathematical analysis or natural experimentation.

*A computer program*, written in one of the *high-level programming languages* (e.g. Fortran, C++, C#, Python, Pascal, etc.), implements an *algorithm* according to which computer simulations are performed.

*Computer simulation* – the use of a computer to represent the dynamic responses of one system by the behavior of another system modeled after it. A computer simulation uses a mathematical model, or description, of a real system in the form of a computer program. This model is composed of equations that duplicate the functional relationships within the real system. When the program is run, the resulting mathematical dynamics form an analog of the behavior of the real system, with the results presented in the form of data. Computer simulations enable researchers to understand complex interplay of a multitude of factors of physical or social environment.

Computer modeling and simulations are used to study the dynamic behavior of systems or objects in response to conditions that cannot be easily or safely applied in real life. They permit researchers to perform exotic experiments or demonstrations without using rare materials or expensive equipment (e.g. modeling nuclear explosion, astrophysical phenomena, pandemics etc.).

For example, in the aeronautical industry, proposed aircraft designs undergo simulated wind-tunnel tests in computer simulations, thus saving hundreds of hours formerly devoted to prototype building and testing.

Time compression is another cost-saving feature of computer simulation technology. Events that can take anywhere from hours to weeks, months or even years in real time can be simulated in a few minutes.

## 2. Algorithm

***Algorithm*** – a set of rules for solving a problem in a finite number of steps. Usually independent of the software or hardware. Any algorithm is required to have the following five features:

**1) *Finiteness*** – an algorithm must always terminate after a finite number of steps.

**2) *Definiteness*** – each step of an algorithm must be precisely defined; the actions to be carried out must be rigorously and unambiguously specified for each case.

**3) *Input*** – an algorithm has zero or more inputs: quantities that are given to it initially before the algorithm begins, or dynamically as the algorithm runs.

**4) *Output*** – an algorithm has one or more outputs: quantities that have a specified relation to the inputs.

**5) *Effectiveness*** – An algorithm is also generally expected to be effective, in the sense that its operations must all be sufficiently basic that they can in principle be done exactly and in a finite length of time by someone using pencil and paper.

The word *algorithm* comes from the Latinized name of the 9th century mathematician Abu 'Abd Allah Muhammad ibn Mūsā al-Khwārizmī (c. 825) — literally, "Father of Abdullah, Mohammed, son of Moses, native of Khwārizm." The Aral Sea in Central Asia was once known as Lake Khwārizm. The Khwārizm region is located in the Amu River basin just south of that sea. Al-Khwarizmī wrote the celebrated book *Kitab aljabr wa'1-muqābala* ("Rules of restoring and equating"); another word, "algebra," stems from the title of his book, which was a systematic study of the solution of linear and quadratic equations.

## 3. Algorithmic thinking

***Science*** *is what we understand well enough to explain to a computer.*
***Art*** *is everything else we do.*

Donald Knuth,

It is instructive to have a look at one of the first algorithm known from the ancient times, Euclid's algorithm.

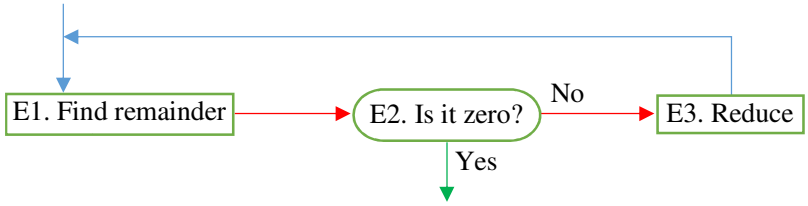
**Euclid's algorithm**. Given two positive integers $m$ and $n$, find their greatest common divisor (GCD), that is, the largest positive integer that evenly divides both $m$ and $n$.

**Algorithm E:**

**E1**. [Find remainder.] Divide $m$ by $n$ and let $r$ be the remainder. (We will have $0 \le r < n$).
**E2**. [Is $r$ zero?] If $r = 0$, the algorithm terminates; $n$ is the answer.
**E3**. [Reduce.] Set $m \leftarrow n$, $n \leftarrow r$, and go back to step **El**. ▌

Of course, Euclid did not present his algorithm in just this manner. The format above illustrates the style in which algorithms are presented. The algorithm has been given an identifying letter (E in this example), and the steps of the algorithm are identified by this letter followed by a number (E1, E2, E3).

Each step of an algorithm, such as step E1 above, begins with a phrase in brackets that sums up as briefly as possible the principal content of that step. This phrase also usually appears in an accompanying flow chart, such as in the figure below, so that the reader will be able to picture the algorithm more readily.




After the summarizing phrase comes a description in words and symbols of some action to be performed or some decision to be made. Parenthesized comments, like the second sentence in step E1, may also appear. Comments are included as explanatory information about that step, often indicating certain invariant characteristics of the variables or the current goals at that step. They do not specify actions that belong to the algorithm, but are meant only for the reader's benefit as possible aids to comprehension.

The arrow "←" in Step E3 is the all-important replacement operation, sometimes called assignment or substitution: "$m \leftarrow n$" means that the value of variable $m$ is to be replaced by the current value of variable $n$. When Algorithm E begins, the values of $m$ and $n$ are the originally given numbers; but when it ends, those variables will have, in general, different values. An arrow is used to distinguish the replacement operation from the equality relation: We will not say, "Set $m = n$," but we will perhaps ask, "Does $m = n$?" The " = " sign denotes a condition that can be tested, the "←" sign denotes an action that can be performed.

The operation of *increasing n by one* is denoted by "$n \leftarrow n + 1$" (read "$n$ is replaced by $n + 1$" or "$n$ gets $n + 1$"). In general, "variable ← formula" means that the formula is to be computed using the present values of any variables appearing within it; then the result should replace the previous value of the variable at the left of the arrow. Persons untrained in computer work sometimes have a tendency to say "$n$ becomes $n + 1$" and to write "$n \rightarrow n + 1$" for the operation of increasing $n$ by one; this symbolism can only lead to confusion because of its conflict with standard conventions, and it should be avoided.

Notice that the order of actions in step E3 is important: "Set $m \leftarrow n$, $n \leftarrow r$" is quite different from "Set $n \leftarrow r$, $m \leftarrow n$," since the latter would imply that the previous value of $n$ is lost before it can be used to set $m$. Thus the latter sequence is equivalent to "Set $n \leftarrow r$, $m \leftarrow r$." When several variables are all to be set equal to the same quantity, we can use multiple arrows; for example, "$n \leftarrow r$, $m \leftarrow r$" may be written "$n \leftarrow m \leftarrow r$." To interchange the values of two variables, we can write "Exchange $m \leftrightarrow n$"; this action could also be specified by using a new variable $t$ and writing "Set $t \leftarrow m$, $m \leftarrow n$, $n \leftarrow t$."

An algorithm starts at the lowest-numbered step, usually step 1, and it performs subsequent steps in sequential order unless otherwise specified. In step E3, the imperative "go back to step E1" specifies the computational order in an obvious fashion. In step E2, the action is prefaced by the condition "If $r = 0$"; so if $r \neq 0$, the rest of that sentence does not apply and no action is specified. We might have added the redundant sentence, "If $r \neq 0$, go on to step E3."

The heavy vertical line "▌" appearing at the end of step E3 is used to indicate the end of an algorithm and the resumption of text.

We have now discussed virtually all the notational conventions used in algorithms.

So much for the *form* of algorithms; now let us *perform* one. It should be mentioned immediately that the reader should *not* expect to read an algorithm as if it were part of a novel; such an attempt would make it pretty difficult to understand what is going on. An algorithm must be seen to be believed, and the best way to learn what an algorithm is all about is to try it. The reader should always take pencil and paper and work through an example of each algorithm immediately upon encountering it in the text. Usually the outline of a worked example will be given, or else the reader can easily conjure one up. This is a simple and painless way to gain an understanding of a given algorithm, and all other approaches are generally unsuccessful.

So this is an algorithm. The modern meaning for algorithm is quite similar to that of *recipe*, *process*, *method*, *technique*, *procedure*, *routine*, *rigmarole*, except that the word "algorithm" connotes something just a little different. As it was noted previously, besides merely being a finite set of rules that gives a sequence of operations for solving a specific type of problem, an algorithm has five important features. Let's see how these features are implemented in Algorithm E.

**1) *Finiteness***. An algorithm must always terminate after a finite number of steps. Algorithm E satisfies this condition, because after step E1 the value of $r$ is less than $n$; so if $r \neq 0$, the value of $n$ decreases the next time step E1 is encountered. A decreasing sequence of positive integers must eventually terminate, so step E1 is executed only a finite number of times for any given original value of $n$. Note, however, that the number of steps can become arbitrarily large; certain huge choices of $m$ and $n$ will cause step E1 to be executed more than a million times. (A procedure that has all of the characteristics of an algorithm except that it possibly lacks finiteness may be called a *computational method*. Euclid originally presented not only an algorithm for the greatest common divisor of numbers, but also a very similar geometrical construction for the "greatest common measure" of the lengths of two line segments; this is a computational method that does not terminate if the given lengths are incommensurable. Another example of a nonterminating computational method is a *reactive process*, which continually interacts with its environment.)

**2) *Definiteness***. Each step of an algorithm must be precisely defined; the actions to be carried out must be rigorously and unambiguously specified for each case. Our algorithms will hopefully meet this criterion, but they are specified in the English language, so there is a possibility that the reader might not understand exactly what the author intended. To get around this difficulty, formally defined *programming languages* or *computer languages* are designed for specifying algorithms, in which every statement has a very definite meaning. An expression of a computational method in a computer language is called a *program*.

In Algorithm E, the criterion of definiteness as applied to step E1 means that the reader is supposed to understand exactly what it means to divide $m$ by $n$ and what the remainder is. In actual fact, there is no universal agreement about what this means if $m$ and $n$ are not positive integers; what

is the remainder of $-8$ divided by $-\pi$? What is the remainder of 59/13 divided by zero? Therefore, the criterion of definiteness means we must make sure that the values of $m$ and $n$ are always positive integers whenever step E1 is to be executed. This is initially true, by hypothesis; and after step E1, $r$ is a nonnegative integer that must be nonzero if we get to step E3. So $m$ and $n$ are indeed positive integers as required.

**3)** ***Input***. An algorithm has zero or more inputs: quantities that are given to it initially before the algorithm begins, or dynamically as the algorithm runs. These inputs are taken from specified sets of objects. In Algorithm E, for example, there are two inputs, namely $m$ and $n$, both taken from the set of positive integers.

**4)** ***Output***. An algorithm has one or more outputs: quantities that have a specified relation to the inputs. Algorithm E has one output, namely $n$ in step E2, the greatest common divisor of the two inputs.

**5)** ***Effectiveness***. An algorithm is also generally expected to be effective, in the sense that its operations must all be sufficiently basic that they can in principle be done exactly and in a finite length of time by someone using pencil and paper. Algorithm E uses only the operations of dividing one positive integer by another, testing if an integer is zero, and setting the value of one variable equal to the value of another. These operations are effective, because integers can be represented on paper in a finite manner, and because there is at least one method (the "division algorithm") for dividing one by another. But the same operations would *not* be effective if the values involved were arbitrary real numbers specified by an infinite decimal expansion, nor if the values were the lengths of physical line segments (which cannot be specified exactly). Another example of a noneffective step is, "If 4 is the largest integer $n$ for which there is a solution to the equation $w^n + x^n + y^n = z^n$ in positive integers $w$, $x$, $y$, and $z$, then go to step E4." Such a statement would not be an effective operation until someone successfully constructs an algorithm to determine whether 4 is or is not the largest integer with the stated property.

Let us try to compare the concept of an algorithm with that of a cookbook recipe. A recipe presumably has the qualities of finiteness (although it is said that a watched pot never boils), input (eggs, flour, etc.), and output (TV dinner, etc.), but it notoriously lacks definiteness. There are frequent cases in which a cook's instructions are indefinite: “Add a dash of salt.” A “dash” is defined to be "less than ⅛ teaspoon," and salt is perhaps well enough defined; but where should the salt be added — on top? on the side? Instructions like "toss lightly until mixture is crumbly" or "warm cognac in small saucepan" are quite adequate as explanations to a trained chef, but an algorithm must be specified to such a degree that even a computer can follow the directions. Nevertheless, a computer programmer can learn much by studying a good recipe book.

Ordinary language is often characterized by ambiguity and uncertainty. Consider the following two sentences:

“Henry is calling Pierre because he got a new book from the publisher.”
“The customer calls the waiter because he hasn't served the steak yet.”

Syntactic analysis alone is not enough to eliminate ambiguity in the first sentence, and semantics cannot help us either. Only the knowledge and understanding of relationships and connections between the elements of this particular reality can give unambiguity and, therefore, meaning to the first sentence. When we have a proper model of reality, which correctly reflects such knowledge, the ambiguity is eliminated. The last sentence is an example of this: Unlike the first sentence, in the second one the same pronoun “he” does not cause confusion.

## 4. Analysis of Algorithms

The issue of complexity of an algorithm is especially important when it comes to tasks of a class where at each stage of the solution there are several options for continuing to the next stage of the solution. In case of a complex, multi-step algorithm, we result in an increasing number of possible

solutions, the investigation of which may require very large computational resources (execution time, memory capacity). This often makes it practically impossible to implement such an algorithm using modern computers. The purpose of algorithm optimization is to find a trade-off between the time required for its implementation and the amount of memory. These two parameters always place opposite requirements on the optimal algorithm. It is intuitively understandable that to reduce the time required to implement the algorithm, we need to increase the amount of random access memory (RAM) to store more intermediate data to avoid computing them at each new step of the algorithm where they will be needed. On the other hand, it is possible to organize the algorithm so as to minimize the requirements for RAM volume. Instead, the implementation time of the algorithm will naturally increase.

---

Simple examples of optimization

***Example 1*** **Calculate $X^{2n}$ so that the number of multiplications is minimal** (***n*** is integer)
Two versions of the algorithm:

[1]: X←$x$ [initialize $x$]
[2]: N←$n$ [initialize $n$]
[3]: X2N←1 [initialize: enter 1 in X2N]
[4]: X2N← X2N*X [Previous content of X2N is multiplied by X;]

- • • •
- • • • [ do 2N times]
- • • •

[2N+3]: X2N← X2N*X [X2N ← $X^{2N} = x^{2n}$] ($2n$ multiplications!)

Calculate the same quontity as follows:
[1]: X←$x$ [initialize $x$]
[2]: N←$n$ [initialize n]
[3]: XN←1 [[initialize: enter 1 in X2N]
[4]: XN← XN*X

- • • •
- • • • [do N times]
- • • •

[N+3]: XN← XN*X [XN ← $X^{N}$ (= $x^{n}$)]
[N+4]: X2N← XN*XN [XN ← X2N (= $x^{2n}$)] ($n$ + 1 multiplications!)

***Example 2*** Calculate $X^{27}$:

[1] X2 ← X*X
[2] X3 ← X2*X
[3] X6 ← X3*X3
[4] X12 ← X6*X6
[5] X13 ← X12* X
[6] X26 ← X13*X13
[7] X27 ← X26*X

This procedure requires only 7 multiplications to calculate $X^{27}$ instead of 26.

***Exercise 1*** Write an algorithm for evaluating a polynomial

$$P_n(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0,$$

i.e. for computing its value for a given value of $x = x_0$.

*Hint:* Use the following expression for the polynomial

$$P_n(x) = \underbrace{(((...((}_{n-1} a_n x + a_{n-1})x + a_{n-2})x + \cdots + a_3)x + a_2)x + a_1)x + a_0$$

Show that direct calculations involve $n(n+1)/2$ multiplications and $n$ additions. Compare these numbers to the numbers of the corresponding arithmetic operations required in your algorithm.

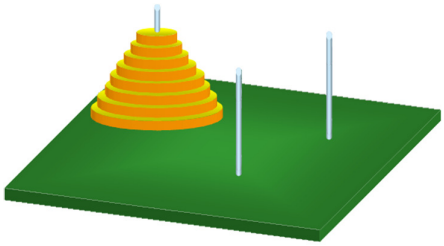
***Exercise 2*** Write an algorithm and a code to solve the quadratic equation

$$ax^2 + bx + c = 0,$$

where $a$, $b$, $c$ are real numbers. Prepare your code to perform both single and double precision calculations.

***Exercise 3*** Prove that the number found by Euclid's algorithm is indeed the greatest common divisor of two positive integers. Compare it to other GCD search algorithms you know.

***Exercise 4*** A puzzle called the Tower of Hanoi, introduced by the French mathematician Edouard Lucas in 1883, states: We are given of a tower of disks, initially stacked in decreasing size on one of three pegs as is shown in figure:



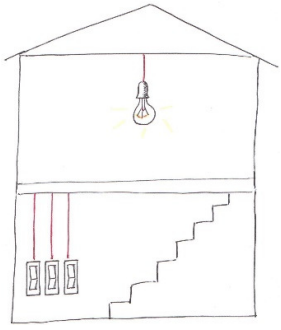
The goal is to move the entire tower onto one of the pegs so that the following two rules are to be observed: 1) move only one disc at a time and 2) never put a larger disc onto a smaller one in this process.

a) In the case of $n$ disks, find how many moves are necessary and sufficient to perform the task.

b) Write down an algorithm for solving the puzzle.

***Exercise 5*** Downstairs in a house are three identical on-off switches. One of them controls the lamp in the attic. The puzzle is to work out which switch controls the lamp.

The rules are as follows. You are allowed to manipulate the switches all you like, and then you are allowed only one trip to the attic. Find an algorithm for solving the puzzle.



***Exercise 6*** Write an algorithm for computing the integrals

$$I_n = \int_0^1 x^n e^{x-1} dx \quad n = 1, 2, \ldots$$

using the recurrence relation $I_n = F(I_{n-1})$, $n = 2, 3. \ldots$, where $I_1 = 1/e$.
Do calculations in single and double precision and compute $I_n$s for $n = 2$ to $n = 20$.
Explain the results.

*Hint*: Find the recursive function $F$ using integration by parts.